

Aula 32 - Testes em Vue.js com Vitest

Duração: 4 horas (1h teórica + 3h prática)

Parte Teórica (1 hora)

1. Por que Vitest?

- **Rápido:** Execução em memória (não precisa de build).
- **Compatível:** API similar ao Jest, mas otimizado para Vite.
- **Suporte nativo a:**
 - Vue 3 Composition API
 - TypeScript
 - ES Modules

2. Comparação Vitest vs Jest

Critério	Vitest	Jest
Performance	Mais rápido (integrado ao Vite)	Mais lento (necessita build)
Configuração	Pronto para Vue 3	Requer plugins adicionais
Watch Mode	Atualização instantânea	Mais lento

3. Estrutura de Testes em Vue

```
plaintext

tests/
├─ unit/
│   ├── components/
│   │   └─ ComponentA.spec.js
│   └─ composables/
│       └─ useFeature.spec.js
└─ e2e/                                     # (Opcional, com Cypress/Playwright)
```

Parte Prática (3 horas)

Atividade 1: Configuração do Vitest (30 min)

1. Instalação:

```
bash

npm install -D vitest @vue/test-utils happy-dom
```

2. Configuração (vite.config.js):

```
javascript

import { defineConfig } from 'vite'
import vue from '@vitejs/plugin-vue'

export default defineConfig({
  plugins: [vue()],
  test: {
    globals: true,
    environment: 'happy-dom'
  }
})
```

3. Script no package.json :

```
json

"scripts": {
  "test": "vitest",
  "test:watch": "vitest watch"
}
```

Atividade 2: Testando Componentes (1h30min)

Componente Sob Teste (Counter.vue)

```
vue

<template>
  <button @click="increment">{{ count }}</button>
</template>

<script setup>
import { ref } from 'vue'
const count = ref(0)
const increment = () => count.value++
</script>
```

Teste Unitário (Counter.spec.js)

javascript

```
import { mount } from '@vue/test-utils'
import Counter from '@components/Counter.vue'

describe('Counter.vue', () => {
  it('increments count on click', async () => {
    const wrapper = mount(Counter)
    await wrapper.find('button').trigger('click')
    expect(wrapper.text()).toContain('1')
  })
})
```

Teste com Props (Greeting.spec.js)

javascript

```
it('renders greeting message', () => {
  const wrapper = mount(Greeting, {
    props: { name: 'John' }
  })
  expect(wrapper.text()).toMatch('Hello, John!')
})
```

Atividade 3: Testando Composables (1h)

Composable Sob Teste (useCounter.js)

javascript

```
import { ref } from 'vue'

export function useCounter(initialValue = 0) {
  const count = ref(initialValue)
  const increment = () => count.value++
  return { count, increment }
}
```

Teste (useCounter.spec.js)

javascript

```
import { useCounter } from '@composables/useCounter'
import { renderHook } from '@testing-library/vue'

describe('useCounter', () => {
  it('increments count', () => {
    const { result } = renderHook(() => useCounter())
    result.increment()
    expect(result.count.value).toBe(1)
  })
})
```

Atividade 4: Mock de APIs (Opcional)

Mock com MSW (Mock Service Worker)

1. Instalação:

```
bash
```

```
npm install -D msw
```

2. Configuração (src/mocks/handlers.js):

```
javascript
```

```
import { rest } from 'msw'

export const handlers = [
  rest.get('/api/products', (req, res, ctx) => {
    return res(ctx.json([{ id: 1, name: 'Notebook' }]))
  })
]
```

3. Teste com Mock (ProductList.spec.js):

```
javascript
```

```
import { setupServer } from 'msw/node'
import { handlers } from '@mocks/handlers'

const server = setupServer(...handlers)

beforeAll(() => server.listen())
afterEach(() => server.resetHandlers())
afterAll(() => server.close())

it('loads products', async () => {
  const wrapper = mount(ProductList)
```

```
    await flushPromises()
    expect(wrapper.findAll('li')).toHaveLength(1)
  })
})
```

Tarefa para Casa

1. Teste de Eventos:

- Criar um teste para um componente que emite eventos (`@emits`).

2. Teste de Roteamento:

- Mockar `vue-router` e testar navegação programática.

Próxima Aula

- Tópico: Projeto Final (Parte 2).
- Atividade prática: Testes E2E com Cypress.

Resultado Esperado

- Cobertura de testes para:
 - Componentes Vue 3.
 - Composables (lógica reativa).
 - Integração com APIs mockadas.

plaintext

- ✓ Vitest configurado
- ✓ Componentes testados
- ✓ Composables validados

<https://vitest.dev/og.png> (Vitest: Rápido e eficiente para Vue 3) 🚀